



Algorithms, Design, and Analysis

Dr. Allam Ameen

Assistant Professor

Helwan International Technological University (HITU)

Electronics Research Institute, Microstrip Department

Spring 2026

Lecture Outcomes

- Learning what is recursive.
- Calculating $T(n)$ for recursive algorithms.
- Calculating complexity of recursive function

Introduction to Recursion

General introduction to Recursion

- ❑ A recursive function is a function that calls itself until a “base condition” is true, and execution stops.
- ❑ Many times, a problem broken down into smaller parts is more efficient. Dividing a problem into smaller parts aids in conquering it. Hence, recursion is a divide-and-conquer approach to solving problems.

Sub-problems are easier to solve than the original problem

- Solutions to sub-problems are combined to solve the original problem.
- “Divide-and-conquer” is most often used to traverse or search data structures such as binary search trees, graphs, and heaps. It also works for many sorting algorithms, like quicksort and heapsort.

Example1 : Factorial of an integer number

```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))  
num = 5  
print(factorial(num))
```

- ❑ Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.
- ❑ The Python interpreter limits the depths of recursion to help avoid infinite recursions, resulting in stack overflows.

```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))  
num = 5  
print(factorial(num))
```

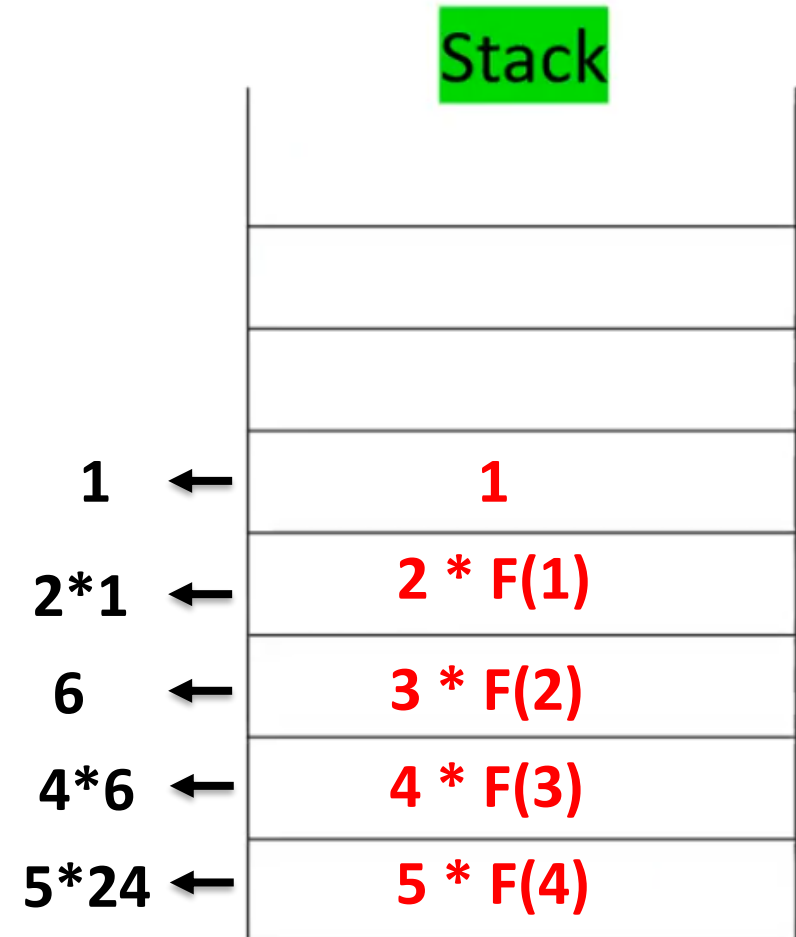
```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))
```

```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))
```

```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))
```

```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))
```

```
def factorial(n):  
    if n== 1:  
        return 1  
    else:  
        return (n * factorial(n-1))  
num = 5  
print(factorial(num))
```



In [2]: |

```
1
2 def fibonacci(n):
3     if(n <= 1):
4         return n
5     else:
6         return(fibonacci(n-1) + fibonacci(n-2))
7
8
9 n = int(input("Enter number of terms:"))
10 print("Fibonacci sequence:")
11 for i in range(n):
12     print(fibonacci(i))
13
14
15
16
17
18
19
20
21
22
23
```

I


```
1
2 def fibonacci(n):
3     if(n <= 1):
4         return n
5     else:
6         return(fibonacci(n-1) + fibonacci(n-2))
7
8
9 n = int(input("Enter number of terms:"))
10 print("Fibonacci sequence:")
11 for i in range(n):
12     print(fibonacci(i))
13
14
15
16
17
18
19
20
21
22
23
```

Console 1/A
Removing all variables...

In [2]: runfile('C:/Users/
ProfSaleh/.spyder-py3/temp.py', wdir='C:/
Users/ProfSaleh/.spyder-py3')

Enter number of terms:7

Fibonacci sequence:

0

1

1

2

3

5

8

In [3]:

```
1
2 def fibonacci(n):
3     if(n <= 1):
4         return n
5     else:
6         return(fibonacci(n-1) + fibonacci(n-2))
7
8
9 n = int(input("Enter number of terms:"))
10 print("Fibonacci sequence:")
11 for i in range(n):
12     print(fibonacci(i))
13
14
15
16
17
18
19
20
21
22
23
```

Console 1/A
Removing all variables...

In [2]: runfile('C:/Users/ProfSaleh/.spyder-py3/temp.py', wdir='C:/Users/ProfSaleh/.spyder-py3')

Enter number of terms:7

Fibonacci sequence:

0

1

1

2

3

5

8

In [3]: runfile('C:/Users/ProfSaleh/.spyder-py3/temp.py', wdir='C:/Users/ProfSaleh/.spyder-py3')

Enter number of terms:40

```
1
2 def fibonacci(n):
3     if(n <= 1):
4         return n
5     else:
6         return(fibonacci(n-1) + fibonacci(n-2))
7
8
9 n = int(input("Enter number of terms:"))
10 print("Fibonacci sequence:")
11 for i in range(n):
12     print(fibonacci(i))
13
14
15
16
17
18
19
20
21
22
23
```

```
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765
10946
17711
28657
46368
75025
121393
196418
317811
514229
832040
1346269
2178309
```

**Calculating $T(n)$ for
recursive algorithms**

Example 1: Calculate T(n)

```
int fact(int n) {  
    if (n == 0)  
        return 1;  
    else  
        return fact(n - 1);  
}
```

Non-Recursive Term

Recursive Term

Answer

$$T(n) = \underbrace{T(n-1)}_{\text{Recursive Term}} + \underbrace{1}_{\text{Non-Recursive Term}}$$

Example 2: Calculate T(n)

```
int fact(int n){  
    if (n == 0)  
        return 1;  
    else  
        return 4 * fact(n - 1);  
}
```

Answer

$$T(n) = \underbrace{4}_{\text{Recursive Term}} \cdot \underbrace{T(n-1)}_{\text{Non-Recursive Term}} + 1$$

Example 3: Calculate T(n)

```
int fact(int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * fact(n - 1);  
}
```

Answer

$$T(n) = \underbrace{n T(n-1)}_{\text{Recursive Term}} + \underbrace{1}_{\text{Non-Recursive Term}}$$

Example 4: Calculate T(n)

```
int fact(int n){  
    if (n == 1)  
        return 1;  
    else  
        return fact( n/2 );  
}
```

Answer

$$T(n) = T(n/2) + 1$$

Recursive
Term

Non-Recursive
Term

Example 5: Calculate T(n)

```
int fact(int n){  
    if (n == 1)  
        return 1;  
    else  
        return fact( n/2 ) + fact( n/2 );  
}
```

Answer

$$T(n) = \underbrace{2T(n/2)}_{\text{Recursive Term}} + \underbrace{1}_{\text{Non-Recursive Term}}$$

Example 6: Calculate T(n)

```
int fact(int n) {  
    if (n == 1)  
        return 1;  
    else  
        return fact( n/2 ) + fact(3n/2 ) ;  
}
```

Answer

$$T(n) = \underbrace{T(n/2) + T(3n/2)}_{\text{Recursive Term}} + \underbrace{1}_{\text{Non-Recursive Term}}$$

Example 7: Calculate T(n)

```
int fact(int n){  
    if (n == 1)  
        return 1;  
    else  
        return fact( n-1 ) + fact( n-2 );  
}
```

Answer

$$T(n) = T(n-1) + T(n-2) + 1$$

Recursive
Term

Non-Recursive
Term

Example 8: Calculate T(n)

```
int fact(int n){  
    if (n == 1)  
    {  
        for(i=1, i<=n; i++)  
        {  
            print(i);  
        }  
        return 1;  
    }  
    else  
        return fact( n-1 ) + fact( n-2 );  
}
```

Non-Recursive Term

Recursive Term

Answer

$$T(n) = \underbrace{T(n-1) + T(n-2)}_{\text{Recursive Term}} + \underbrace{\sum_{i=1}^n 1}_{\text{Non-Recursive Term}}$$

Complexity of Recursion Algorithms

Simplifying recurrences

obtain a formula with no recurrence.

□ There are four methods for solving Recurrence:

- Iteration Method
- Substitution Method
- Recursion Tree Method
- Master Method

$$n! = 1 * 2 * 3 * 4 * \dots * n$$

$$1! = 1$$

$$n! = n * (n-1)!$$

$$(n-1)! = (n-1)! * (n-2)!$$

Complexity of Recursive Algorithms - The Iteration Method

In this method, we first convert the recurrence into a summation. We do so by iterating the recurrence until the initial condition is reached.

- ❑ Convert the recurrence into a summation and try to bound it using known series
- ❑ Iterate the recurrence until the initial condition is reached.
- ❑ Use back-substitution to express the recurrence in terms of n and the initial (boundary) condition.

Complexity of Recursive Algorithms - The Iteration Method

Example 1:

```
func Sum(int n ) int {  
    if n == 1 {  
        return 1  
    }  
    return n + Sum(n-1)  
}
```

$$\begin{aligned}T(n) &= T(n-1) + 1 \\T(n-1) &= T(n-2) + 1 + 1 \\T(n-2) &= T(n-3) + 1 + 1 + 1 \\T(n-3) &= T(1) + \underbrace{1 + 1 + \dots + 1}_{(n-1) \text{ times}}\end{aligned}$$

Complexity **$O(n)$**

Complexity of Recursive Algorithms - The Iteration Method

Example 2: $T(n) = c + T(n/2)$

$$T(n) = c + T(n/2)$$

$$= c + c + T(n/4)$$

$$= c + c + c + T(n/8)$$

$$T(n/2) = c + T(n/4)$$

$$T(n/4) = c + T(n/8)$$

Assume $n = 2^k$

$$T(n) = c + c + \dots + c + T(1)$$

$$= c \lg n + T(1)$$

$$= O(\lg n)$$

Complexity

$O(\log_2 n)$

Example 3:

$$T(n) = n + 2T(n/2)$$

$$\begin{aligned} T(n) &= n + 2T(n/2) \\ &= n + 2(n/2 + 2T(n/4)) \\ &= n + n + 4T(n/4) \\ &= n + n + 4(n/4 + 2T(n/8)) \\ &= n + n + n + 8T(n/8) \\ \dots &= in + 2^iT(n/2^i) \\ &= kn + 2^kT(1) \\ &= n\lg n + nT(1) = O(n\lg n) \end{aligned}$$

$$T(n/2) = n/2 + 2T(n/4)$$

Assume: $n = 2^k$

Complexity

$O(n\log_2 n)$

Example 4:

$$\begin{aligned} T(n) &= 1 \text{ if } n = 1 \\ &= 2T(n-1) \text{ if } n > 1 \end{aligned}$$

$$\begin{aligned} T(n) &= 2[2T(n-2)] = 2^2 T(n-2) \\ &= 4[2T(n-3)] = 2^3 T(n-3) \\ &= 8[2T(n-4)] = 2^4 T(n-4) \end{aligned}$$

$$T(n) = 2^i T(n-i)$$

$$n-1=1 \Rightarrow i=n-1$$

$$T(n) = 2^{n-1} T(1)$$

$$T(n) = 2^{n-1}$$

Expand the recurrence and express it as a summation of terms of n and initial condition

Complexity $O(2^n)$

Complexity of Recursive Algorithms تعقيد الخوارزميات العودية

Substitution method

□ In the substitution method, instead of trying to find an exact closed-form solution, we only try to find a closed-form bound on the recurrence

Substitution method: guess the solution and then prove its correctness by induction.

two-steps solution:

1. Guess the exact form of the solution: Typically, one would precompute a set of numerical results for different input n 's and derive a formula.
2. Prove the correctness of the guess: Mathematical induction is the tool of choice in most cases.

The method is very effective provided we can complete the step 1 correctly.

Substitution Method

Example1:

Show that the solution of $T(n) = T(n - 1) + n$ is $O(n^2)$

We guess $T(n) \leq cn^2$

$$\begin{aligned} T(n) &\leq c(n-1)^2 + n \\ &= cn^2 - 2cn + c + n \\ &= cn^2 + n(1-2c) + c \leq cn^2, \end{aligned}$$

Where the last step holds for $c > \frac{1}{2}$

$$O(n^2)$$

Example 2:

Show that the solution of $T(n) = T(\lceil n/2 \rceil) + 1$ is $O(\lg n)$

Guess $T(n) \leq c \lg(n - a),$

$$\begin{aligned} T(n) &\leq c \lg(\lceil n/2 \rceil - a) + 1 \\ &\leq c \lg((n + 1)/2 - a) + 1 \\ &= c \lg((n + 1 - 2a)/2) + 1 \\ &= c \lg(n + 1 - 2a) - c \lg 2 + 1 \quad (c \geq 1) \\ &\leq c \lg(n + 1 - 2a) \quad (a \geq 1) \\ &\leq c \lg(n - a), \end{aligned}$$

The ceiling function gives an integer number value which is a numeric value greater than the value of the function.

The floor function gives an integer number value which is a numeric value lesser than the value of the function

Example 3:

Show that the solution of $T(n) = 2T(\lfloor n/2 \rfloor) + n$ is $O(n \lg n)$.

Guess: $T(n) \leq cn \lg(n)$

$$\begin{aligned} T(n) &\leq 2c \lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n + (1 - c)n \\ &\leq cn \lg n, \end{aligned}$$

Some Substitution complexities

$$T(n) = O(1) + T(n-1) \quad T(n) = O(n)$$

$$T(n) = O(n) + T(n-1) \quad T(n) = O(n^2)$$

$$T(n) = O(1) + T(n/2) \quad T(n) = O(\log n)$$

$$T(n) = O(n) + T(n/2) \quad T(n) = O(n)$$

An almost-rule-of-thumb:

- The solution is maximum recursion depth times amount of work in one call

Divide-and-conquer algorithms

$$T(n) = O(n) + 2T(n/2) \quad T(n) = O(n \log n)$$

Any
Questions ?

